

Scalable Parallel-in-Time Integration for Equations of Motion

Nathaniel W. Chapman

Department of Computer Science
Central Washington University

2025-08-10

Table of Contents

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

Content

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

It's About Time

The main issue

Calculations take time...and we only have so much.

Minimizing Time

- Total runtime = $\frac{\text{time}}{\text{calculation}} * \text{number of calculations}$
- Want to minimize runtime
- Minimizing time / calculation $\iff$
Speeding up sequential evaluation
- Maximizing calculations / time $\iff$
Adding more parallelization

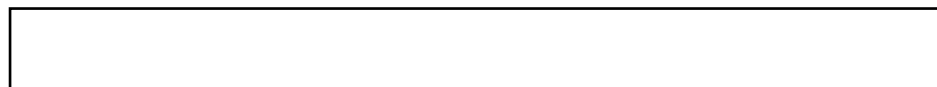


Figure 1:

Parallelizing Time

- Physics depends on time and space
- Space has been parallelized
- Time has remained sequential
- Parallel-in-Time integration offers a new avenue to speedup



Figure 2:

Maximizing Resource Utilization

- Space discretized as much as it can
e.g. constrained by CFL
- Spatial parallelization saturated
- Remaining cores sit idle

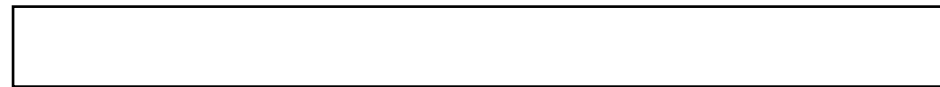


Figure 3:

More Threads is Always the Answer

- PinT scales with threads
- CPUs have ~10 threads
- GPUs have ~10,000 threads
- Distributed has ∞ threads



Figure 4:

This work

- Implement the Parareal Algorithm
- Use GPUs and distributed systems
- Simulate motion

Content

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

Growth of Parallel-in-Time Integration

- PinT is growing
- Not a new idea
- More cores $\Rightarrow$ more interest
- On track for more growth

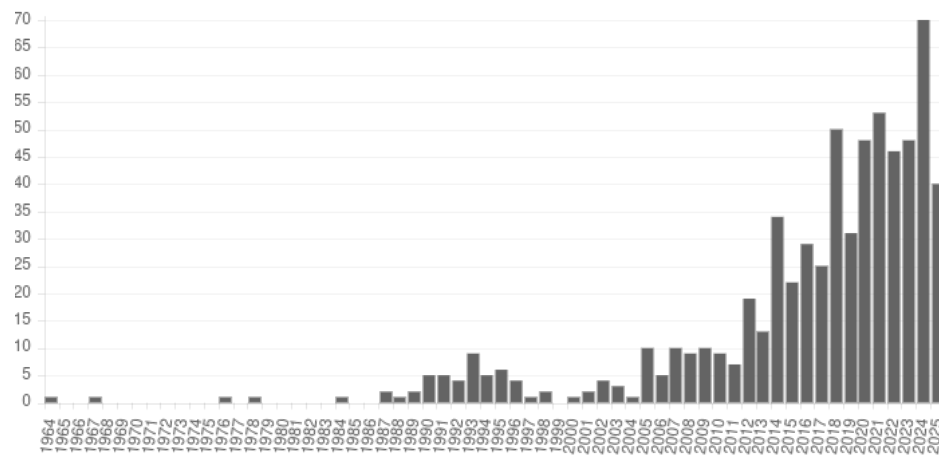


Figure 5: The number of papers published using parallel-in-time integration has been growing and even accelerating over the years.

Applications of Parallel-in-Time Integration

- Applications in many sciences
- **Physics:** E&M, Plasmas, Fluids, Materials, Astrophysics
- **Engineering:** Manufacturing
- **Operations:** Optimal control
- **Economics:** Game theory, Market dynamics
- **Biology:** Animal patterns, Blood flow in fish
- **Machine Learning:** Training neural networks

Methods of Parallel-in-Time Integration

- Specialized methods
- **General:** Parareal, PITA, PFASST, RIDC
- **Hyperbolic:** ParaDiag, MGRIT
- **Parabolic:** STMG, WRMG

Diagonalization-based Parallel-in-Time Methods ParaDiag: diagonalization-based Parallel-in-Time (PinT) algorithms, which can handle both both dissipative and hyperbolic equations	Multigrid Reduction in Time (MGRIT) Multigrid Reduction in Time algorithm developed at the LLNL	PFASST PFASST is a time-parallel expansion of spectral deferred corrections methods.
PITA The parallel implicit time-integrator.	Parareal Parareal is one of the most widely studied time-parallel methods.	Revisionsit Integral Deferred Correction (RIDC) RIDC Want your method to show up here? You can do it yourself!
Space-time Multigrid (STMG) STMG is a time-parallel method for parabolic PDEs.	Space-time concurrent multigrid waveform relaxation (WRMG) WRMG is normally only a space-concurrent method; time parallelism is possible using cyclic reduction.	

Figure 6: Several parallel-in-time methods have been developed. Each method aims to address limitations of others as well as providing new approaches in general.

Implementations of Parallel-in-Time Integration

- Different languages with different means of parallelism
- **Languages:** Fortran, C, C++, Python
- **Parallelism:** OpenMP, MPI
- **HPC:** Very few

PFASST++ A C++ library for SDC, MLSDC and PFASST.	PararealF90 Fortran90 special-purpose Parareal code.	PinT-TF A performance and convergency testing framework for Parallel-in-Time methods, currently only for Parareal.
PyMGRIT PyMGRIT is a package for the Multigrid-Reduction-in-Time (MGRIT) algorithm in Python.	SWEET Development written in C++ which allows to study PinT time integration and compare it to other time integration methods using global spectral methods in space.	XBraid A C library for the multigrid reduction in time (MGRIT) approach.
libridc A C++ library for RIDC.	pySDC A Python prototyping framework for SDC-based integration routines.	Want your code to show up here? You can do it yourself!

Figure 7: Several parallel-in-time methods have been implemented. Implementations have been developed in a variety of languages with a variety of approaches to parallelism.

Language of Parallel-in-Time Integration

- Traditional HPC languages have been used e.g. C/C++, Fortran
- Python has been used
- Julia has the speed of C with the ease of Python
- Julia natively supports parallel, GPU, and distributed
- PinT should be implemented in Julia

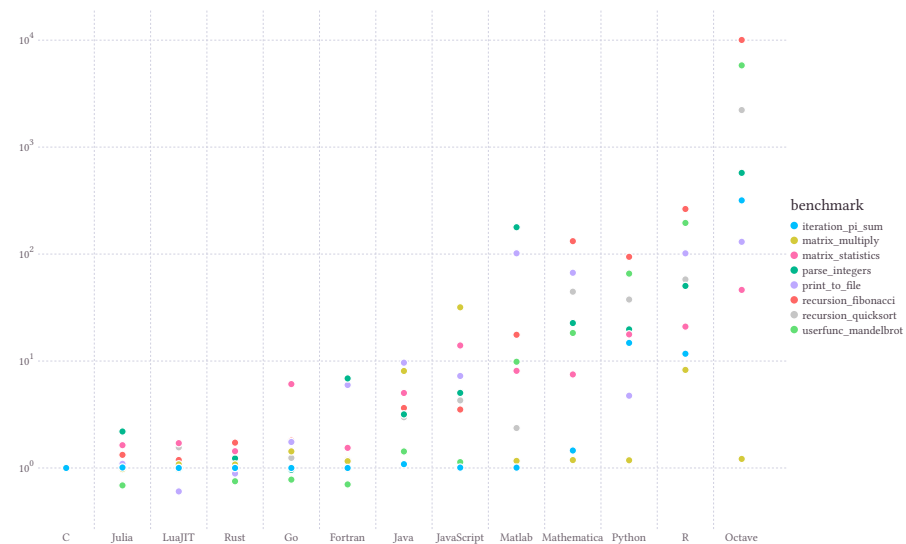


Figure 8: Several parallel-in-time methods have been implemented. Implementations have been developed in a variety of languages with a variety of approaches to parallelism.

This Contribution to Parallel-in-Time Integration

- **Application:** Physics of motion
- **Method:** Parareal
- **Parallelism:** Distributed GPUs
- **Language:** Julia

Content

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

Review of Equations of Motion

Content

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

Review of High-Performance Computing

Parareal on the GPU

Content

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

Numerical Analysis

Latency & Data Transfers

Content

Introduction

Background

The Parareal Algorithm

The Parareal Algorithm at Scale

Performance Analysis

Conclusion

Thanks for Listening.